

COURSE REVIEW NOTES

Introduction to AI and Machine Learning on Google Cloud

A plain-language synthesis of the arguments in each module.
Follows the original course order.

Six modules · foundations, generative AI, build options, workflow, and wrap-up

How to use this document

This is a study aid, not a transcript. It restates the *arguments* of Introduction to AI and Machine Learning on Google Cloud in everyday language so you can review the logic without flipping through every slide. Labs, YAML, and click-paths are left out on purpose. When you need the exact how-to, go back to the original lesson.

Product names in this course have been shifting (Vertex AI is often called Agent Platform; image models appear as Nano Banana). The arguments below use the names as taught, and focus on *why you would pick one tool over another*.

What “argument” means here

Each module is treated as a claim: a problem exists, a Google Cloud approach answers it, and there are trade-offs. Read for the decision, not for a product catalog.

Contents

00 Course introduction — AI is a tool, not a mystery
01 AI foundations — predict vs create, then the Google stack
02 Generative AI — models, prompts, grounding, agents
03 AI development options — meet people where they are
04 AI development workflow — kitchen to table, then automate
05 Course wrap-up — the same story, shorter
Related AI Hypercomputer one-pager (extra file in the folder)

00 Course introduction

Module 0 · Why this course exists

The opening claim is emotional, then practical: AI feels everywhere and changes every week (first generative AI, then agents). That overwhelm is compared to a kitchen full of gadgets. **AI is just another powerful tool. You do not need to be a genius to pick it up.**

The course promises a path from “huh?” to “I can build that.” Module 1 is ingredients and tools (foundations, infrastructure, BigQuery ML). Module 2 is generative AI: Gemini, Veo (video), Nano Banana (images), prompts, multimodality, and agents that not only write but act. Module 3 is how you actually build: no-code Gemini Enterprise apps, AutoML, pre-trained APIs, BigQuery ML, custom training. Module 4 is the whole workflow: prepare data, develop the model, serve it.

In one sentence. You can start without being a coding genius; the rest of the course is which kitchen tool to pick for which job.

01 AI foundations

Module 1 · Predict vs create, then the Google stack

The module's spine is a food-truck company, Coffee on Wheels, helped by a digital firm called Data Beans. Three business pains: where to park the trucks, how many sales to expect, and how to run marketing without a huge team. The argument: **AI has moved from “make us a bit more efficient” to “let us invent new ways of working,” and you usually need both prediction and generation in the same product.**

The use case as a map

The demo dashboard mixes weather, events, street view, forecasts, and campaign copy with pictures. Under the hood: multimodal Gemini for intake, BigQuery for analytics, Agent Platform for ML, agents that call apps, Looker and Maps for people to see and act. Input is text, photos, video. Process is forecast plus generate. Output is insight you can click into a route or an email.

Predictive vs generative

- **Predictive AI** (also called traditional or discriminative) looks at what already happened and classifies or forecasts. Sales next week. Will this visitor buy. Route given traffic.
- **Generative AI** makes new stuff and can take action: copy, images, summaries, a chatbot that talks to a sales team.
- There is no hard wall. Forecast churn (predictive), then a chatbot that helps the sales team explore those scores (generative). Find segments, then write a campaign for each segment.
- Pick by outcome: numbers and labels → predictive. Multimodal content and automation → generative. Often both.

Why Google, and the three layers

Google's pitch: it already runs Search, Maps, and Workspace on AI; it ships Gemini and Agent Platform; it talks about responsible AI (bold, but not reckless). The cloud itself is three layers:

- **Infrastructure** — compute, network, storage. This course only needs you to know it exists.
- **Development** — Agent Platform from design to deploy, with BigQuery next door. This is the main playground of the course.
- **Applications** — ready-made tools for people who are not engineers, so they can prototype.

Infrastructure in one page

Google has been in data since 1998 and sold cloud since 2008. Under the hood: networking and security; then compute and storage, **decoupled so they can grow separately** (unlike a laptop); then data and AI products. Compute ranges from “I want the VM” (Compute Engine) through GKE to “just run my container” (Cloud Run). Chips: CPUs and GPUs are general; **TPUs** are Google's custom chips for matrix-heavy ML, more efficient for that job. Storage: Cloud Storage for files and media; BigQuery, AlloyDB, Cloud SQL, Spanner for tables; BigQuery can also query JSON and even point at files in Cloud Storage.

The data-to-AI path is always: ingest (Pub/Sub, Dataflow, Spark, Data Fusion) → store and analyze (the databases above plus Looker) → activate with AI (Agent Platform, AutoML, notebooks). BigQuery and Agent Platform talk to each other: SQL in BigQuery can train a model; a notebook can pull BigQuery data.

What “AI” and “ML” actually mean here

AI is the umbrella (machines doing things that look intelligent). Machine learning is the subset where the computer learns from examples instead of being given every rule. Deep learning stacks hidden layers. Generative AI uses huge foundation models (often large language models) to produce content and act on requests.

- **Supervised:** labeled data, task-driven. Classification (cat vs dog, logistic regression) or regression (sales number, linear regression).
- **Unsupervised:** no labels, find structure. Clustering (k-means segments), association (beer next to diapers), dimensionality reduction (PCA to fewer features).
- Quiz the course loves: predict spending from history → supervised regression, linear. Segment customers with no pre-named groups → unsupervised clustering, k-means.

BigQuery ML: stay in SQL

Classic ML is a loop: load data, try models, retune, deploy. BigQuery ML's claim: **if your data already lives in the warehouse, do not export it to a separate ML island.** BigQuery is storage plus a fast SQL engine, scaled independently on Google's network. Phases: load (and join other sources), pick features (it will one-hot encode categories for you), CREATE MODEL, MLEVALUATE (accuracy, precision, recall, ROC-AUC), MLPREDICT.

Start simple (logistic / linear) as a benchmark before DNNs and XGBoost, which cost more to train. Other built-ins: k-means, ARIMA+ forecasting, matrix factorization, anomaly detection, AutoML Tables, importing TensorFlow for batch, exporting for online serving. The lab predicts whether a store visitor will buy, with Gemini helping write SQL.

***In one sentence.** Decide predict vs create (or both); put data in BigQuery; train the first model with SQL before you reach for a custom neural net.*

02 Generative AI

Module 2 · Models, prompts, grounding, agents

Generative AI **makes content and takes action** from a prompt: text, code, images, speech, video, even 3D. Chatbots were the first wave. Agents are the next: book travel, run a workflow, draft a diagnosis suggestion. Google's story starts with the Transformer (2017), Gemini as a native multimodal model (2023), then an agent platform for enterprises.

Three-layer gen AI stack

- **Foundation models** — Gemini (general, multimodal), Veo (video), Nano Banana (images), embeddings (search-friendly vectors), plus media specialists like Chirp (voice) and Lyria (music).
- **Development** — Agent Studio, Agent Garden, Model Garden, later ADK and Agents CLI.
- **Applications** — Gemini Enterprise app and Gemini Notebook for people who should not have to code.

Foundation models

A foundation model is large: many parameters, huge training data, expensive compute. More parameters generally means more capacity. Inside Gemini: **Pro** for hard reasoning, **Flash** for fast chatty work, **Flash-Lite** for cheap high-volume jobs (batch translation, summaries). Gemini can take a cookie photo and return a recipe video — that mix of senses is why it beats a single-purpose model when your problem is mixed (photos + weather + PDFs + disaster video for insurance risk).

Horizontal models are general (content, summaries, chat). Vertical models are smaller specialists fine-tuned on industry data (diagnose, price a risk). Pre-training is school; fine-tuning is medical school. You talk to models three ways: console (no code), APIs (low code), SDKs in notebooks (code).

Idea to app: three people, one platform

Cymbal Insurance: Bea (analyst, no code, wants a prototype), Ann (developer, wants to craft prompts), Ian (ML engineer, wants pipelines and tuning). Gemini Enterprise app is the employee workspace; Agent Platform is for builders; Gemini Enterprise for CX is for customer-facing industries. **Agent Studio** is the workshop: prototype, tune, ground with live data, deploy with generated code.

What a good prompt is

Prompt design is writing the request. Prompt engineering is the loop of design → refine → measure. A prompt has a **task** (required), **context** / system instructions (optional), and **examples** (optional — few-shot). Zero-shot is task only. Structure matters: order, labels, delimiters. Be specific, iterate from simple, avoid jargon. The best of three sample prompts is the one with role, steps, and a template — not “write a risk report.”

Studio helps: “Help me write,” a prompt gallery, multimodal attachments (PDFs, YouTube, GCS, Drive), prompt templates with variables (natural-language functions), and one-click “deploy as app” on Cloud Run. Then you leave the toy prototype and engineer for real.

Parameters: temperature, top-K, top-P

The model is always guessing the next token from a probability list. Always picking the top word is dull; picking at random can be nonsense. **Temperature** low = typical answers (Q&A, summaries); high = more surprise (creative copy). **Top-K** samples from the K likeliest words (can pick a weird word if the top is 80% and the second is 10%). **Top-P** grows the set until probabilities add up to P, so the set size adapts. You do not have to twiddle K and P every day. Compare prompts, models, and settings side by side; score against a ground-truth answer; save versions in prompt management.

Grounding vs tuning

Pretrained models go stale. **Grounding** is the *what*: pin answers to trusted data (Google Search now, or your own corpus). **RAG** is a common *how*. Tuning is different: it changes (or lightly adapts) the model’s insides. Prompt design changes no weights. Parameter-efficient / adapter tuning updates a small slice. Full fine-tuning updates everything — best quality, most compute. Agent Studio’s supervised fine-tune uses JSONL pairs of `input_text` / `output_text` (hundreds of labeled examples) for classification, summarization, extraction, chat. The result lands in Model Registry and can be served from an endpoint.

From chatbot to agent to agentic AI

A foundation model cannot see your claims database or send the email. An **AI agent** can: it has a goal, a model as brain, tools (APIs — GET/POST as hands and senses), and an orchestration layer as nervous system (decide → act → observe → decide again). **Agentic AI** is several agents coordinated (insurance + underwriting + claims).

Where to build: Model Garden is the brain shelf. Developers use Agent Studio (low-code loops), ADK (graphs, agent-to-agent, sandboxes), Agent Runtime (scale, observability, long-running jobs). Business users use Gemini Enterprise app (search across company docs, agent hub, Agent Designer, Notebook as research buddy) or CX templates. When you need legacy systems, policy tone, and custom behavior, the no-code app is not enough — move to Agent Platform.

Decision tree in plain words: no-code ready-made (Enterprise app / CX) → low-code templates (Studio, Garden) → pro-code (ADK or build your own). Ease vs flexibility is the only axis that matters.

In one sentence. Pick a model for the job, write a prompt with task/context/examples, ground it in real data, and only then add agents if the work must reach other apps.

03 AI development options

Module 3 · Meet people where they are

Module 2 was generative. This module asks: **how do you build predictive models — and how do you choose a path that matches the person and the data, not the fanciest demo?** Gartner's sting: about half of enterprise ML projects never leave the pilot. Pain is production (scale, monitor, retrain) and ease (too much coding, no one workflow).

The menu

- **No-code apps:** Gemini Enterprise app and CX — business users.
- **No/low-code AutoML:** point and click on your data; Google searches architectures.
- **Low-code APIs:** Google already trained the model; you pass text or an image.
- **SQL in BigQuery ML:** data already in the warehouse; analysts stay in SQL.
- **Custom training / ADK:** full control, more skill, more time.

How to choose

Look at data type, how much labeled data you have, how much ML skill, whether you must tune hyperparameters, and how long you can wait.

- No training data, common perception task (vision, language, speech) → pre-trained APIs.
- Tabular data in BigQuery and SQL fluency → BigQuery ML.
- Your own data, little coding, want a custom model → AutoML.
- Need the architecture, framework, and training loop in your hands → custom training on Workbench or Colab Enterprise.

AutoML and APIs do not let you twiddle every hyperparameter. Custom training and BigQuery ML do. APIs take no train time. Custom training usually takes the longest. Budget matters; check current prices.

Agent Platform as the workshop

One place to prepare data, train, deploy, and manage — for both gen AI and predictive AI. Pipeline in words: upload (GCS, BigQuery, laptop) → Feature Store → train and tune → deploy and watch. Four S's they use: seamless, scalable (MLOps grows compute/storage), sustainable (reuse artifacts), speedy (less code than piecing tools yourself).

What AutoML is actually doing

AutoML (2018 idea, later inside Vertex/Agent Platform) exists because humans waste months on preprocess → try models → retune. Four phases: auto preprocess (TensorFlow Transform: quantiles, embeddings, one-hot, n-grams, datetime splits); neural architecture search plus transfer learning (do not start from zero — same idea as fine-tuning an LLM); bag an ensemble of the top models (often about ten, budget-dependent) instead of betting on one; then predict. You still click: dataset → train → online or batch prediction.

Pre-trained APIs

Good custom models want huge clean data. If you do not have it, call a finished model like a function: auth, pick model, send input, read output. You do not care how the kitchen works. Google offers Gemini/foundation APIs, Agent Platform APIs, plus speech, vision, documents, conversation. Gemini is eating a lot of the older single-task APIs. Natural Language API still teaches the pattern: entities, sentiment, syntax, categories — demo in the browser, then JSON + cURL (or an SDK) in production.

Custom training

Use it when AutoML's black box is not enough. Choose a **pre-built container** (furnished kitchen: Python, TensorFlow, PyTorch) or a **custom container** (empty room: you pick machines and disks). Code in Workbench (managed Jupyter for the whole data-science loop) or Colab Enterprise. Libraries (TensorFlow, PyTorch, scikit-learn) mean you do not write linear algebra from scratch. TensorFlow stack: hardware (CPU/GPU/TPU) → low-level ops → layer libraries → Keras. Typical Keras loop: Sequential layers → compile (loss, optimizer, metrics) → fit. JAX is Google's newer high-performance numeric library for research and production.

***In one sentence.** Match the tool to data, skill, and control: API if you have no data, SQL if the data is already in BigQuery, AutoML if you want a custom model without coding, custom training only when you must own the loop.*

04 AI development workflow

Module 4 · Kitchen to table, then automate

Building a model is compared to a restaurant. The argument: **the three stages never really finish, and a model that never ships is just a theory**. AutoML (UI) and Pipelines (code) walk the same stages.

Three stages

- **Data preparation** — upload and engineer features. Quality and quantity decide how well the machine learns. Data may stream or batch; structured (tables) or not (images, video).
- **Model development** — train, evaluate, train again.
- **Model serving** — deploy and monitor. If serving never happens, there is no product.

The loop is not a straight line. Training may send you back to invent better features. Monitoring may show drift and force a retrain. That is what MLOps is for.

Features and Feature Store

A feature is a column that helps the prediction — the carrot you peel before cooking. Feature Store is a shared pantry: one place to build, serve, and reuse features for training and for live traffic, batch or real-time, at low latency. It also holds embeddings for gen AI and “find similar items.” Typical online path: data in BigQuery → optional feature groups → a feature view into the online store → serve. Share, reuse, scale, simple UI.

Training and judging the meal

In the UI you pick dataset, objective (classification, regression, forecast), AutoML vs custom, target column, which features, a node-hour budget, maybe early stopping. AutoML then searches. You judge classification with a **confusion matrix**:

- True positive / true negative: the model was right.
- False positive (Type 1): cried cat and it was not.
- False negative (Type 2): missed the cat.
- **Recall** = of the real positives, how many did we catch (TP / TP+FN).
- **Precision** = of the predicted positives, how many were real (TP / TP+FP).

Precision and recall trade off. Catch all spam → high recall (some good mail dies). Never block a real letter → high precision (some spam gets through). The confidence threshold slides that trade-off: high threshold = picky (precision up, recall down). Feature importance bars tell you which columns actually drive the answer — drop the

rest.

Loan-risk lab, in words: a perfect true-positive rate on “will repay” means you never reject a good customer. An 13% false-positive rate on defaulters means some bad loans still get approved. Threshold 0: everyone is “safe,” half actually default. Threshold 1: almost nobody is approved, but those who are really do repay. Pick a threshold that matches the cost of each mistake.

Serving

- **Online / endpoint:** low latency, on demand (recommendations while the user browses). Must deploy to an endpoint.
- **Batch:** no rush (a fortnightly campaign). Hit the model resource; no endpoint required.
- **Edge:** on the factory camera when cloud round-trips are too slow, private, or offline.

MLOps

DevOps for models: data and code both keep moving. Practice means automate and watch every step — continuous integration, **continuous training**, continuous delivery. Agent Platform Pipelines is serverless orchestration. Use TFX if you already live in TensorFlow at huge tabular scale; otherwise Kubeflow Pipelines. A pipeline is components (functions) in two worlds: dev/test (extract → analyze → prepare → train → evaluate → validate → registry) and prod (predict + monitor). Prefer Google’s pre-built ops; write a custom component only for a special gate (for example: deploy only if metric $\geq$ threshold).

Maturity: Phase 0 is all UI (you must be able to do it by hand first). Phase 1 automates pieces (a training job). Phase 2 stitches CI/CT/CD. Example: create tabular dataset → AutoML train → custom eval gate → endpoint → deploy — compile, submit, reuse a template.

Optional: how a network actually learns

ANNs (and then DNNs, CNNs, RNNs, LLMs) pass numbers through layers. Weights are what training discovers. Each neuron: weighted sum of inputs (+ bias), then an **activation**. Without activation, many layers collapse to one linear formula — useless for hard problems. ReLU (zero if negative, else keep the number) is a common activation. Training compares $\hat{y}$ to y , then nudges weights. That is enough theory for this course; skip the lesson if you already know it.

***In one sentence.** Prepare features once and share them, train and judge with precision/recall on purpose, serve online or in batch, then automate the loop or the model rots in a notebook.*

05 Course wrap-up

Module 5 · The same story, shorter

The last module is a recap, not a new argument. It walks the same path: Coffee on Wheels showed predict + generate; Google’s toolbox is infrastructure / development / apps; infrastructure is compute, storage, and a data-to-AI pipeline; models are supervised vs unsupervised; BigQuery ML is SQL-in-place; gen AI creates and acts; the gen stack is models → Studio → apps; prompts need task, context, examples; agents are brain + tools + orchestration; pick no-code, AutoML, APIs, or custom training; the workflow is prepare → develop → serve, by UI or by pipeline.

Closing advice: apply this to your own work. More training lives at cloud.google.com/training/machinelearning-ai; certifications at cloud.google.com/certifications.

***In one sentence.** If you remember only the recap, remember the choice: predict or create, no-code or full control, and never stop at a model that is not served.*

Related file: AI Hypercomputer — a closer look

From the course one-pager, plus Google Cloud documentation (Sept 2026)

The folder includes a two-page recap from another path (AI Infrastructure: Introduction to AI Hypercomputer). The course itself only names the stack. Google’s public docs treat Hypercomputer as the **unified front door to AI infrastructure** — not a single button, but hardware, software, and a way to buy capacity, designed together so accelerators spend time computing instead of waiting. Google’s word for that useful time is **goodput**.

That is the argument: renting 100 GPUs is not a win if they sit idle on I/O, on a missing node, or on a network hop. Hypercomputer is Google trying to sell the whole machine Google uses internally for training and serving, not just a VM with a GPU attached.

Official starting points: cloud.google.com/ai-hypercomputer/docs/overview and the *consumption, GPU, networking, and storage* pages next to it.

The three layers

- **Performance-optimized infrastructure.** NVIDIA GPUs and Cloud TPUs, plus the network and storage that keep them fed.
- **Open software.** Tuned PyTorch, JAX, and TensorFlow, plus a place to run them: GKE, Slurm (via Cluster Director), or Cluster Toolkit blueprints.
- **Consumption options.** How you obtain the machines: on-demand, Spot, Flex-start, reservations, calendar blocks, and Dynamic Workload Scheduler so a whole training job appears at once.

Docs say this vertical stack exists for the whole lifecycle: a small experiment, a giant training run, and low-latency serving. Use cases they list include distributed gen-AI training, gen-AI inference, fraud detection, and recommenders.

GPUs: “general” vs “clustered”

This is the most useful split in the current docs. It is not about the brand of chip first. It is about **how Google treats the machines as a system**.

- **General GPUs** are independent computers. Maintenance can happen one VM at a time. You use ordinary Compute Engine and GKE APIs and a normal VPC (TCP/IP over gVNIC). Good for prototypes, RAG, single-node serving, and training that does not need eight GPUs to breathe in sync.
- **Clustered GPUs** are one tightly coupled fabric of (potentially) thousands of accelerators. Maintenance is synchronized. Placement is dense so the network stays short. Cluster Director is in the picture. This is for foundation-model training, multi-host inference, and heavy reasoning / RL jobs where a straggler node wastes the whole job.

Kind	Typical machines (plain names)	When you want this
General	G2 (L4) for everyday inference and RAG; G4 for cheaper graphics-ish inference; A2 (A100) for strong single-node serving or small fine-tunes; A3 High with 1–4 H100s; A3 Edge for multi-host inference over a normal VPC.	You are serving, experimenting, or training small. You can live with ordinary cloud networking.
Clustered	A3 High with 8 H100s, A3 Mega (H100, faster network), A3 Ultra (H200), A4 (B200), A4X (GB200), A4X Max (GB300) for huge / bare-metal training.	The job is a big train or a big multi-host serve. GPUs must talk to each other all day.

Watch utilization (CUDA / Tensor cores) and, on clusters, goodput recipes: what fraction of time was real compute versus waiting. Clustered hardware also gets health checks and proactive node replacement; general GPUs lean on ordinary node auto-repair so serving pods stay up.

TPUs, in the same picture

A TPU is not a faster GPU. It is a chip that only wants matrix math — the multiply-and-add heart of neural nets. Inside: a systolic array (thousands of multipliers wired to their neighbors) and on-chip high-bandwidth memory so bigger models and batches fit. Host RAM feeds an infeed queue; results come back on an outfeed queue. You scale by joining chips into **slices** (and multi-slice) with little or no code change. You reach TPUs from Compute Engine, GKE, or Vertex AI — same Hypercomputer umbrella, different amount of toil.

The network is half the computer

On clustered machines, GPU-to-GPU traffic and “everything else” (disks, Cloud Storage, the internet, SSH) use **different physical NICs and different VPCs** so a checkpoint write does not steal bandwidth from training. Titanium / gVNIC handles the front-door (north–south) path. GPU fabrics handle east–west.

- **Rail-aligned topology** (clustered): hosts in a sub-block share a top-of-rack switch (often one hop). A block is several sub-blocks (at most two hops). A cluster is many blocks (one extra hop). Metadata about block and sub-block is exposed so the scheduler can place a job next to itself.
- **GPUDirect RDMA (RoCE)** — A4X Max, A4X, A4, A3 Ultra: GPU memory talks almost directly to the NIC. A4/A3 Ultra can do on the order of 3.2 Tbps GPU-to-GPU. These types typically need **three VPCs** (two ordinary + one RDMA profile).
- **GPUDirect-TCPXO** — A3 Mega: TCP offload; about double A3 High’s network.
- **GPUDirect-TCPX** — A3 High (8 GPU) and A3 Edge: payload goes GPU → NIC with fewer copies. A3 High often wants four GPU VPCs; A3 Mega eight.
- **Plain TCP/IP** — A2, G4, G2, N1. Fine for general GPUs. Do not mix RDMA with NCCL Fast Socket / TCPX plugins on the same cluster.

NCCL is the library GPUs use for “all reduce” and friends. After you build a cluster, Google tells you to run NCCL tests before you trust a week-long train. On GKE, pods get multiple interfaces so RDMA and storage stay apart.

Storage: keep the chips fed

Docs (aimed at clustered GPUs) treat storage as two families: a huge object store, and a POSIX parallel file system.

- **Cloud Storage** is the cheap, durable lake. **Rapid Bucket** is a zonal, high-I/O class (docs quote up to about 15 TB/s and sub-millisecond opens). **Rapid Cache** is an SSD read cache in the same zone (up to about 2.5 TB/s) so you do not recode apps; useful when data lives in a multi-region bucket. **Cloud Storage FUSE** mounts a bucket like a disk (parallel downloads, client cache). **Hierarchical namespace** makes folders real (atomic

rename, faster lookup, much higher QPS) — recommended for heavy checkpointing; required for Rapid zonal buckets.

- **Managed Lustre** is a managed POSIX parallel file system (docs: sub-ms, up to about 10 TB/s, tens of millions of IOPS/TiB, up to ~80 PB). No FUSE dance. Cluster Director can stand up Lustre or Filestore for you.

Job	First pick	Why, in plain words
Prep / store giant training sets	Cloud Storage Rapid Bucket	Huge, durable, cheaper. Put it in the same zone as the GPUs.
Synchronous checkpoints, RL weight blasts, KV-cache offload	Managed Lustre	Thousands of workers can hit the same file without falling over.
Async / multi-tier checkpoints	Rapid Bucket (+ FUSE or fsspec)	Good enough if you tune the client; cheaper than Lustre as a lake.
Load weights for inference (cold start)	Rapid Cache or Rapid Bucket	Warm the model in-zone so the first request is not a download marathon. vLLM + Run:ai Model Streamer is the serving hint they give.

How you actually stand it up

The course one-pager listed four landings. Current cluster docs (especially for A3 High 8-GPU through A4X) say this:

- **Compute Engine** — you own the orchestrator. Maximum control, maximum homework (VMs, MIGs, bulk insert).
- **Cluster Director** — fully managed **Slurm**. Google wires compute, network, and Filestore/Lustre. Built for admins and researchers who want to submit jobs, not debug fabrics. Needed for the biggest clustered types as a managed path.
- **Cluster Toolkit** — open-source blueprints (Terraform-style) for Slurm or GKE. You can edit the blueprint. Best-practice default without going fully managed.
- **GKE** — if your team already thinks in pods. Standard or Autopilot. Blueprints can turn on GPUDirect RDMA by default. COS node image is required for RDMA; Ubuntu/Windows are not. A4X / A4X Max node pools must use reservation-bound provisioning.
- **Vertex AI / Agent Platform** (the course’s “fully managed” row) — Google hides the cluster. Less of the knobs; you still sit on the same chips and consumption models when you train or serve at scale.

The decision is the same as the rest of the course

Direct GCE if you must. GKE if you already run Kubernetes. Cluster Toolkit if you want a known-good cluster without inventing VPC math. Cluster Director if you want Slurm and no fabric project. Vertex if the model job should look like an API, not a cluster.

How you pay: do not buy GPUs like VMs

Accelerators are scarce. On-demand “create VM now” often fails or costs list price. Dynamic Workload Scheduler exists because a training job needs **every** interconnected node at the same time — seven of eight is a wasted day. Flex-start queues until the whole block is free, then runs for a set duration (docs: up to seven days, no minimum).

Option	You get	Typical use	Catch
On-demand	Start now if stock exists. Pay list. Runs until you stop it.	Serving, tiny experiments.	Worst odds of getting hot GPUs. No discount.
Spot	Cheapest (docs: up to ~91% off). Starts if spare capacity exists.	Jobs that can die and resume: batch, some HPC.	Google can preempt any time.
Flex-start (DWS)	You wait. Then a dense block runs up to 7 days. Often ~53% off on A2/A3/A4/G4.	Fine-tunes, small pre-trains, batch inference, HPC sims.	Not for “must start at 9:00 Monday.” A4X / A4X Max are out.
Standard reservation	Capacity is yours. CUDs can lower the rate. No extra quota burn.	Always-on production inference on general GPUs.	You pay for idle if the fleet is quiet.
Future reservation (capacity block)	A dated block of clustered GPUs, dense, high assurance. CUDs for ≥1 year.	Pre-training a foundation model; multi-host serving.	You plan with the account team. When the window ends, VMs stop or delete.
Calendar mode	Like booking a hotel: up to 90 days, request in advance, dense clustered GPUs (A4 / A3 Ultra / Mega / High-8). Docs cap around 80 VMs. ~53% off.	A known training or fine-tune window.	Google must approve. Not open-ended.

Rule of thumb from the docs: general GPUs → Flex-start or Spot beat on-demand for obtainability and price; clustered monsters → reserve a block. Calendar is the short reserved stay; future capacity blocks are the long one.

A practical chooser

- Notebook, RAG demo, L4/A100 serving → general GPU, on-demand or a small reservation.
- Fine-tune that can start tonight or tomorrow → Flex-start.
- Sweep that can restart from a checkpoint → Spot (and design for preemption).
- Multi-week foundation train on H200/B200/GB200 → clustered + future/calendar reservation + RDMA network + Lustre or Rapid for checkpoints.
- Always-on customer-facing inference → reservation (or Vertex endpoints) so a Spot blip does not page you.
- Team already on Kubernetes → GKE (+ Toolkit blueprint). HPC/Slurm shop → Cluster Director.
- You do not want to see VPCs → Vertex / Agent Platform custom training and serving.

In one sentence. Hypercomputer is chips plus fabric plus a reservation strategy: pick general vs clustered, keep GPU traffic off the storage NIC, and buy capacity the way the job actually fails — not the way you buy a web VM.

How the arguments connect

Read as one story: AI is a tool (00). Real businesses mix forecasts with generated content on top of BigQuery and a three-layer cloud (01). Generative work is a model + a prompt + optional grounding, and becomes an agent only when it must use other systems (02). You pick a build path by skill and data, not by fashion (03). Every path still walks prepare → train → serve, and production is the part that usually dies (04). The wrap-up is that map again (05). Hypercomputer is the hardware chapter when the model is too big for “a GPU in a notebook.”

A short checklist while you revise

Is the job predict, create, or both? Is the data already in BigQuery? Do you have labels — or should you call an API? Is Gemini the multimodal brain, or do you need Veo/Nano Banana? Does the prompt have task, context, and examples? Do you need grounding more than fine-tuning? Does the work require tools (an agent) or just text? Precision or recall — which mistake is more expensive? Online, batch, or edge? If this must run next year, who retrains it? If you need accelerators: general or clustered? Flex-start, Spot, or a reservation? Rapid Bucket or Lustre for checkpoints?

This synthesis is for personal review. It is not official Google documentation. Hypercomputer details follow public docs as of September 2026 and will move as machine types and discounts change. Confirm prices and quotas in the console before you plan a training window.